BITCOIN TRANSACTION INTELLIGENCE

FeePilot

Offline Fee Awareness, Payment Continuity and Transaction Recovery

Updated project documentation evolving the ConfirmShield transaction-recovery concept with the Offline Sats idea.

DOCUMENT	VERSION	DATE
Product concept, architecture, safety and MVP plan	2.0 — FeePilot update	October 2026

Core idea
Help Bitcoin users and merchants understand fee conditions, make informed decisions using recent cached fee data when offline, and recover eligible unconfirmed transactions after reconnecting.

Product principle: offline estimates are useful context, not live network data. FeePilot must show when information was last refreshed and validate against current conditions when connectivity returns.

1. Executive Summary

FeePilot is a Bitcoin transaction intelligence and recovery assistant. It combines the strongest parts of the original ConfirmShield concept—transaction analysis, recovery-strategy comparison, authorized RBF/CPFP workflows and status tracking—with an offline-first fee-awareness layer inspired by the Offline Sats idea.

When online, FeePilot retrieves fee estimates and relevant mempool-policy information from a configured Bitcoin data source and saves a timestamped snapshot locally. When connectivity is unavailable, the merchant can still view that snapshot, understand its age, and receive a recommendation clearly labelled as based on cached data. Once online again, FeePilot refreshes the estimates and checks transaction status before recommending or performing any broadcast or recovery action.

FeePilot does not make Bitcoin broadcast or confirmation possible without connectivity. Offline mode supports local planning, record keeping and use of previously retrieved information. It does not prove that a payment has been broadcast, received by the network, or confirmed.

Product goals

- **Prevent where possible:** help users choose a reasonable fee before broadcasting, based on recent information and stated urgency.
- **Explain uncertainty:** show the timestamp, age and limitations of cached estimates.
- **Recover when needed:** analyze unconfirmed transactions and compare supported RBF, CPFP and external-accelerator options.
- **Support disrupted connectivity:** let merchants create local payment/order records and reconcile them after reconnecting.
- **Keep users in control:** require explicit authorization for transactions that spend funds, replace transactions or incur third-party charges.

Primary users: everyday Bitcoin users, merchants and service providers, and technically capable users who need transaction diagnostics and fee/recovery guidance.

2. Problem Statement

Bitcoin transactions can remain unconfirmed when their fee rate is low relative to demand, when they depend on other unconfirmed transactions, or when node and miner policies differ. Users may not know what fee to choose, why a transaction is delayed, or which recovery path is feasible.

Merchants can also experience intermittent internet access. During an outage, they may need to keep records of orders and payment attempts. If FeePilot has recently downloaded fee information, it can still display those saved estimates offline; however, the fee market may have changed by the time a transaction is eventually broadcast.

FeePilot addresses both sides of the problem: **fee awareness before broadcast** and **recovery guidance after a transaction becomes delayed**. It reduces complexity but does not control miners, guarantee inclusion, or replace a wallet's signing authority.

3. Product Scope

In scope for the MVP

- Transaction lookup by txid and display of confirmation state, observed fee rate and available mempool context.
- Online fee-snapshot refresh and local caching with timestamp, network identifier and estimate targets.
- Offline fee-awareness screen that shows cached estimates, their age, selected urgency and freshness warnings.
- Fee calculation from fee rate and estimated transaction virtual size (vsize), with clear distinction between sat/vB and total sats.
- Local-first order/payment-attempt queue with unique local IDs and an explicit 'Awaiting verification' state.
- Synchronization and duplicate-safe reconciliation of local records when connectivity returns.
- Rules-based assessment of RBF, CPFP and external accelerator options, including cost and feasibility where known.
- Monitoring for pending, replaced and confirmed states, with appropriate handling of uncertainty and reorgs.
- One end-to-end RBF or CPFP demonstration on Bitcoin Core regtest.

Out of scope for the MVP

- Guaranteeing confirmation or claiming that a cached fee is the current network fee.
- Broadcasting ordinary on-chain transactions while all devices are offline.
- Treating a locally recorded order, QR code, txid supplied by a user, or offline receipt as proof of payment.
- A full offline Bitcoin or ecash protocol, including offline double-spend prevention.
- Mining simulation, mining-pool operations, or guaranteed miner inclusion.
- Automatic signing/spending without explicit user authorization.
- Supporting every wallet, transaction type, node policy or accelerator provider from day one.

4. Core Features

Feature	Purpose	Expected behavior
Fee Snapshot Cache	Remember recent fee conditions.	Fetch fee estimates online; store network, timestamp, targets and available policy data locally.
Offline Fee Advisor	Help users plan during outages.	Recommend from cached data based on urgency; label stale data and avoid presenting it as live.
Transaction Analysis	Explain current transaction state.	Inspect txid, confirmations, fee rate, mempool visibility and dependencies where available.
Recovery Strategy Engine	Compare recovery paths.	Evaluate RBF, CPFP and available accelerator routes against feasibility, cost, urgency and budget.
Offline Order Queue	Preserve merchant workflow.	Create local records and mark them Awaiting Verification until network checks are possible.
Sync & Reconciliation	Safely resume online operation.	Sync queued records idempotently; refresh fees and verify transaction status.
Confirmation Tracking	Report outcomes over time.	Track pending, replaced and confirmed states; explain observation limits and reorg risk.

5. End-to-End Workflow

A. While online

- FeePilot queries a configured Bitcoin Core node or approved data source for fee estimates at multiple confirmation targets.
- It saves the response, retrieval timestamp, network, units and any available mempool/relay-policy context.
- The user can set a preferred urgency and budget; FeePilot may precompute understandable options for later offline use.

B. While offline

- The interface clearly indicates Offline mode and the time the snapshot was last refreshed.
- It chooses a cached estimate based on the user's urgency and product rules, then displays a freshness warning appropriate to the snapshot age.
- It estimates total transaction fee only when transaction vsize is known or estimated; otherwise, it displays a range or says size is not yet known.
- The merchant may create a local order/payment-attempt record. The record remains Awaiting Verification and is not treated as a settled Bitcoin payment.

C. When connectivity returns

- Refresh fee estimates and current node/mempool policy information.
- Synchronize local records with unique IDs and idempotency checks to avoid duplicate processing.
- Inspect the actual transaction if a txid or raw transaction is available; distinguish not found by this node from proof that no network copy exists.
- Compare the transaction's actual fee rate with current estimates and supported node policy.
- Warn if the fee may be insufficient; offer wait, rebuild, RBF or CPFP guidance where eligible.
- Require explicit authorization before signing/broadcasting or paying an external accelerator, then track the outcome.

6. Understanding Fee Rate, Total Fee and Freshness

Fee rate is commonly expressed in satoshis per virtual byte (sat/vB). The total network fee is the fee rate multiplied by transaction virtual size. For example, if a transaction is estimated at 200 vB and the selected fee rate is 12 sat/vB, the estimated fee is 2,400 sats. The numbers here are illustrative only.

Estimated total fee = fee rate (sat/vB) × transaction vsize (vB)

FeePilot should not confuse a minimum relay fee with a fee rate likely to confirm quickly. Node policy thresholds and confirmation estimates serve different purposes. Estimates depend on the data source, target and current conditions; they are not promises.

Freshness state (example policy)	UI behavior
Recently refreshed	Show cached recommendation and timestamp; still note that conditions can change.
Older than the configured freshness window	Show caution and explain reduced confidence; recommend refreshing before broadcast where possible.
Very stale or missing snapshot	Show historical data only or no recommendation; ask the user to reconnect before treating a rate as suitable.

Freshness state (example policy)	UI behavior
High urgency with stale data	Prominently recommend a fresh check before broadcast when possible; do not claim a guaranteed confirmation time.

Freshness windows must be configurable and tested. A snapshot becoming old does not prove its fee is too low; it means FeePilot has less confidence that it reflects current conditions. A fixed percentage uplift is not a reliable substitute for fresh data.

7. High-Level Technical Architecture

Component	Responsibility
API layer	Accept txids, preferences, order records and requests; return analysis, recommendations and job status.
Bitcoin data adapter	Read transactions, blocks, mempool context and fee estimates from Bitcoin Core or a carefully selected provider.
Fee snapshot service	Fetch, validate, timestamp and persist fee snapshots; preserve network and unit information.
Local cache / offline store	Keep snapshots and local order records available without connectivity; manage expiry and schema versions.
Offline fee estimator	Select a cached estimate based on urgency and freshness rules; calculate total fee when vsize is known.
Analysis service	Normalize transaction facts and assess supported eligibility conditions.
Strategy engine	Compare RBF, CPFP, accelerator and wait/no-action options using explainable rules.
Wallet/signing boundary	Construct unsigned candidates or hand off to a wallet/authorized signer; do not store private keys in the backend.
Provider adapters	Normalize accelerator eligibility, pricing, submission and response formats while retaining provider-specific limits.
Sync & tracking worker	Reconcile queued records, poll status, detect replacements/confirmations and handle retries idempotently.
Database	Store jobs, timestamps, fee snapshots, status history and necessary preferences with data minimization.

Suggested MVP stack

- **Backend:** Rust with Axum, Tokio and SQLx; PostgreSQL for server-side records if needed.
- **Local-first storage:** SQLite for a desktop/local service, or browser storage for a web prototype; select based on deployment model.
- **Bitcoin integration:** Bitcoin Core RPC, initially tested on regtest; use scoped credentials and explicit network configuration.
- **Testing:** Bitcoin Core regtest and Polar for repeatable transaction, fee-bump and sync scenarios.
- **Frontend:** a simple responsive interface for fee snapshots, offline state, transaction analysis, recovery options and history.

Suggested snapshot fields

network; fetched_at; target confirmation estimates; fee-rate units; minimum relay fee if available; mempool minimum fee if available; data source; retrieval success/error; schema version. Estimates may be absent, so fields should be optional and errors handled explicitly.

Illustrative Rust data shape

```
struct FeeSnapshot { network: String, fetched_at: i64, economy_sat_vb: Option, medium_sat_vb: Option,
fast_sat_vb: Option, min_relay_sat_vb: Option, mempool_min_sat_vb: Option, }
```

This is an illustrative schema, not production-ready code. Implementation must map the selected Bitcoin Core version's RPC response fields and units correctly.

8. Recovery Strategy Logic

Route	Use when	Constraints
RBF	A valid replacement can be constructed and authorized.	Replacement policy and fee requirements apply; required inputs must be signed. The original may remain visible to some nodes.
CPFP	A suitable spendable output from an unconfirmed parent can be spent.	Requires control of the output and a signature; package relay and miner policies vary.
External accelerator	A provider accepts the transaction and offers a supported route.	Provider access, pricing, eligibility and pool participation vary; submission is not a guarantee.
Wait / no action	Recovery is unavailable, too costly or not justified.	Explain why no route is recommended and let the user reassess after fresh data is available.

FeePilot's engine should recommend only routes supported by available evidence. If information is missing—such as wallet signing authority, spendable outputs, provider acceptance or a fresh fee estimate—it must communicate the uncertainty rather than imply that a route is available.

9. Security, Safety and Reliability

- **No confirmation promises:** miners make inclusion decisions; fee estimates and accelerator submissions are not guarantees.
- **Explicit authorization:** never broadcast a replacement or child transaction, spend funds or pay a provider without the appropriate consent.
- **Private-key safety:** keep private keys out of the backend MVP; use wallet signing or an explicit signer interface.
- **Offline truthfulness:** a local order, saved fee estimate, QR display or manually entered txid is not proof of broadcast, receipt or confirmation.
- **Freshness and provenance:** display snapshot timestamp, network, source and warning state; prevent mainnet/testnet/regtest mix-ups.
- **Fee transparency:** distinguish network fees from external service charges and show total costs before consent.
- **Transaction validation:** verify network, transaction structure, inputs, outputs, fee calculations and applicable replacement requirements.
- **Sync safety:** use stable local IDs, idempotency keys, retry handling and conflict states to avoid duplicate order processing.
- **Observation limits:** a transaction absent from one node's mempool may still be known elsewhere; report 'not seen by this source' rather than 'does not exist'.
- **Reorg handling and privacy:** track confirmation depth where feasible, minimize logs and identifiers, rate-limit public endpoints and define retention policies.

10. Assumptions and Limitations

- Fee estimates are time-sensitive and probabilistic. Cached information cannot reveal the live fee market while disconnected.
- The first release supports a narrow, tested set of transaction and wallet scenarios rather than universal compatibility.
- A txid can support public transaction lookup but cannot authorize RBF or CPFP.

- RBF requires a valid replacement and signing authority; CPFP requires a suitable spendable output and signing authority.
- External accelerator integrations depend on provider terms, API access, fees and eligibility.
- Regtest is repeatable for demos but does not reproduce real mainnet fee-market pressure by itself.
- Offline Sats in this version means offline-first records and cached information—not a full offline ecash protocol or trustless offline settlement.

11. Hackathon MVP Roadmap

Phase	Deliverable	Acceptance criteria
1 — Fee cache	Online fee snapshot and local persistence.	Fetches estimates for configured targets; stores timestamp, network and units; handles missing estimates.
2 — Offline advisor	Offline fee screen with urgency and freshness state.	Shows saved values offline, computes age, explains uncertainty and calculates total fee when vsize is known.
3 — Offline queue	Local order/payment-attempt records.	Records survive refresh/restart; each has a unique ID and remains Awaiting Verification until checked.
4 — Reconnect & verify	Refresh, synchronize and inspect txids.	Updates snapshots; sync is idempotent; distinguishes not seen by one node from confirmed absence.
5 — Recovery demo	Rules-based RBF/CPFP recommendation and one regtest flow.	Explains eligibility and costs; authorized test transaction can be broadcast and tracked.
6 — Optional accelerator	One provider integration if access permits.	Verifies terms/eligibility, displays service fee, obtains consent and reports provider response accurately.

Recommended short-hackathon priority

Build a polished offline fee-awareness experience and demonstrate safe reconciliation with a Bitcoin Core regtest transaction. Add one recovery route only after the cache, timestamping, fee calculations and reconnect behavior work reliably. A real mainnet accelerator integration is a bonus, not a dependency.

12. Demo Script

- Start a Bitcoin Core regtest node and connect FeePilot.
- Fetch fee estimates and show the timestamped snapshot.
- Simulate loss of connectivity; show that FeePilot still displays cached data and clearly marks it as cached.
- Create an offline order/payment-attempt record and confirm its state is Awaiting Verification.
- Restore connectivity; refresh fee information and synchronize the record without duplication.
- Analyze a controlled unconfirmed test transaction and show a transparent recommendation: wait, RBF, CPFP or unavailable.
- Authorize the selected test recovery action through a test wallet, then track pending/replaced/confirmed status.

13. Success Metrics

- Percentage of online refreshes that produce a valid, timestamped snapshot.
- Correctness of fee units and total-fee calculations against test vectors.
- Correct freshness labels for simulated snapshot ages.
- Percentage of offline order records successfully reconciled without duplicate processing.
- Usefulness and explainability of transaction analysis and recovery recommendations.
- Successful completion of the regtest recovery demonstration.
- Time to generate analysis, and user comprehension of network fees versus provider charges.

- Accuracy of tracked state transitions, including replacement and confirmation changes.

14. Immediate Next Steps

- Define supported networks and the exact RPC methods/fields used for fee snapshots.
- Set initial freshness policies and write tests for recent, stale, missing and failed snapshots.
- Choose the local persistence approach based on whether the first build is web, desktop or mobile.
- Define the offline order schema and reconciliation/idempotency behavior.
- Build a regtest suite for low-fee transactions and one controlled RBF or CPFP path.
- Document the wallet/signing boundary and ensure private keys never enter the service backend.
- Evaluate an external accelerator only after confirming API access, pricing, eligibility and terms.

Closing Statement

FeePilot evolves ConfirmShield from a recovery-first service into a more complete fee-awareness and transaction-assistance workflow. It helps users make better-informed fee choices before broadcast, preserves merchant workflow during connectivity outages, and provides recovery guidance if a transaction remains unconfirmed. Its differentiator is not offline access to live Bitcoin data; it is the combination of useful cached estimates, honest uncertainty, safe reconciliation, authorized recovery and clear status tracking.

Technical note: the exact behavior of fee estimation, relay policy, replacement and package relay depends on the Bitcoin Core version, node configuration and current network conditions. Recommendations should be tested against the supported environment and clearly identify their assumptions.